

Seven Seas A-33

Security Audit

March 7, 2025

Version 1.0.0

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled 'Source Code'. The security audit was performed by the Macro security team on March 4th, 2025.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Medium	1	-	-	1

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Seven Seas team.
- Available documentation in the repository.

Source Code

The following source code was reviewed during the audit:

- **Odos Decoder** (from [PR 146](#))

Commit Hash: 080322f15d61fe0b35b960384ac918723a2a7f7f

Source Code	SHA256
boring-vault/src/base/DecodersAndSanitizers/Protocols/OdosDecoderAndSanitizer.sol	ceb52276010eb1a98914e606c5b22f08d33da9c051718844eed905677ee32dc3

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

M-1

swapCompact not working with addressList

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:

- How bad things can get (for a vulnerability)
- The significance of an improvement (for a code quality issue)
- The amount of gas saved (for a gas optimization)

2. The high/medium/low **likelihood** of the issue:

- How likely is the issue to occur (for a vulnerability)

3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

M-1

swapCompact not working with **addressList**

TOPIC	STATUS	IMPACT	LIKELIHOOD
Assembly	Fixed ↗	Medium	Medium

The **OdosDecoderAndSanitizer** implements the **swapCompact** function that provides a gas-efficient way of swapping. One way to reduce the **calldata** size is to cache addresses in an **addressList**, allowing users to specify only the ID instead of the entire address. However, in **swapCompact**, the logic to mark addresses fails and breaks the decoder.

For addresses that need to be fetched from the **addressList**, assembly is used to store a marker as follows:

```
default {  
    // Store a marker in the high bits and the index in the low bits  
    // We'll use this later to know we need to look up from addressList  
    result := or(0x100000000000000000000000000000000, sub(inputPos, 2))  
    newPos := add(currPos, 2)  
}
```

Reference: [OdosDecoderAndSanitizer.sol#L55](#)

Subsequently, it is checked via **isAddressMarked** if a marker is set and whether an address needs to be fetched:

```
function isAddressMarked(address addr) private pure returns (bool) {  
    return uint256(uint160(addr)) >= 2*160;
```

```
}
```

Reference: [OdosDecoderAndSanitizer.sol#L55](#)

The marker is stored at `0x1000000000000000000000000000000000000000000000000000000000000000` which is the equivalent to `2**156`. However, `isAddressMarked` checks against `≥ 2**160`. Consequently, marked addresses will never be fetched from the `addressList`, breaking the integration to the protocol.

Remediation to Consider

Change the way how relevant addresses are being marked. One possible solution is to simply use a `mapping(address→bool)` to store addresses that need to be fetched from the `addressList`.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.